REACT · STATE YÖNETIMI · DERS 2

Redux ve Redux Thunk

Global state ve asenkron işlemler

HATIRLATMA

Önceki derste: useReducer

dispatch

Değişiklik isteğini bir action olarak gönderir.

reducer

(state, action) alır, yeni state'i döndürür.

Saf kural

State doğrudan değiştirilmez, yenisi üretilir.

Bugün aynı fikri bileşenden çıkarıp tüm uygulamaya taşıyoruz.

Bugünkü yol haritası

01

Redux

Tek store, action ve reducer

02

react-redux ve DevTools

Store'u React'e bağlamak ve izlemek

03

Redux Thunk

Asenkron işlemler ve API çağrıları

04

Özet ve sonraki adım

Karşılaştırma, tekrar ve Redux Toolkit'e geçiş

Veri uzaktaki bileşene nasıl ulaşır?



Buna prop drilling denir. Veri, aradaki bileşenlerin hiçbir ihtiyacı olmadan zincir boyunca taşınır.

Redux’ın üç ilkesi

01

Tek kaynak

Uygulamanın tüm state’i tek bir store nesnesinde tutulur.

02

State salt okunur

State’i değiştirmenin tek yolu bir action göndermektir.

03

Saf reducer

Değişiklikler, eski state ve action’dan yeni state üreten saf fonksiyonlarla yapılır.

Beş temel kavram

Kavram	Ne işe yarar	Örnek
Store	Tüm state'in tutulduğu nesne	createStore(counter)
Action	Ne olduğunu anlatan düz nesne	{ type: "counter/incremented" }
Reducer	Eski state ve action'dan yeni state üretir	counter(state, action)
Dispatch	Action'ı store'a gönderir	store.dispatch(action)
Selector	Store'dan veri okuyan fonksiyon	(state) ⇒ state.count

Redux ile ilk store

```
import { createStore } from "redux";
const initialState = { count: 0 };
function counter(state = initialState, action) {
  switch (action.type) {
    case "counter/incremented":
      return { ...state, count: state.count + 1 };
    default:
      return state;
  }
}
const store = createStore(counter);
store.subscribe(() => console.log(store.getState()));
store.dispatch({ type: "counter/incremented" }); // { count: 1 }
```

Reducer'ın aynısı. Fark: state artık bir component'in değil, store'un içinde.

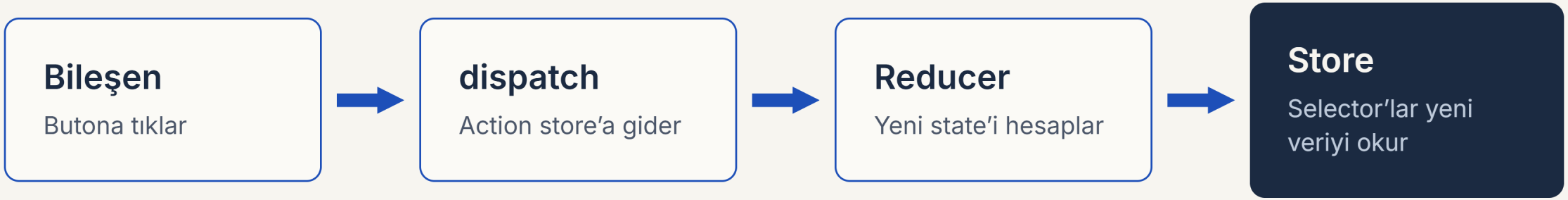
Redux’ı React’e bağlamak

Redux kendi başına React bilmez. react-redux paketi bu köprüyü kurar.

```
// 1) Uygulamayı store ile sarmala (main.jsx)
<Provider store={store}><App /></Provider>
```

```
// 2) Bileşende kullan
function Counter() {
  const count = useSelector((s) => s.count);
  const dispatch = useDispatch();
  return (
    <button onClick={() => dispatch({ type: "counter/incremented" })}>
      {count}
    </button>
  );
}
```

Redux’ta veri akışı



Store değişince, o veriyi kullanan bileşenler yeniden çizilir. Bileşen state’i asla doğrudan değiştirmez.

Redux DevTools

Action geçmişi

Gönderilen her action sırayla listelenir.

State farkı

Her action sonrası state'in nasıl değiştiği görünür.

Zaman yolculuğu

Geçmiş bir adıma dönüp ekranı o state ile görürsünüz.

Tarayıcıya Redux DevTools eklentisini kurun, store'a şu satırı bağlayın:

```
const
store = createStore(counter, window.__REDUX_DEVTOOLS_EXTENSION__?.());
```

Ne zaman Redux, ne zaman değil?

Redux kullan

- Birçok bileşen aynı state'i kullanıyor
- Güncellemeler sık ve karmaşık
- State geçmişini izlemek istiyorsun
- Büyük ekip, ortak bir yapı gerekiyor

Gerek yok

- Tek bileşene özel state (useState yeter)
- Form input değerleri
- Basit açık/kapalı durumları
- Sadece sunucu verisini önbelleğe almak

ALIŞTIRMA 2 · REDUX

Alışveriş sepeti yapalım

1. cart reducer’ı ve store’u oluşturun.
2. Uygulamayı Provider ile sarın.
3. useSelector ile ürün sayısını gösterin, useDispatch ile ekleyin ve çıkarın.

Süre: 30 dakika

Bonus: Toplam fiyatı selector ile hesapla

BÖLÜM 2 · REDUX THUNK

Reducer asenkron iş yapamaz.

Reducer saf ve senkron olmalı. Ama veri çekmek zaman alır: önce "yükleniyor", sonra "başarılı" ya da "hata" bilgisi gönderilmeli. Bu adımları nerede yöneteceğiz?

Redux Thunk nedir?

```
// Normal action creator: bir nesne döner
const increment = () => ({ type: "counter/incremented" });
// Thunk: bir fonksiyon döner
const fetchUsers = () => async (dispatch, getState) => { ... };
```

dispatch

İstediğin zaman, istediğin kadar action gönder.

getState

Mevcut state'i okuyup karara göre davran.

async

API çağrısı gibi asenkron işler burada yapılır.

Thunk ile veri çekmek

```
const fetchUsers = () => async (dispatch) => {
  dispatch({ type: "users/loading" }); // 1. yükleniyor
  try {
    const res = await fetch("/api/users");
    const data = await res.json();
    dispatch({ type: "users/loaded", payload: data }); // 2. başarılı
  } catch (err) {
    dispatch({ type: "users/failed", payload: err.message }); // 3. hata
  }
};
```

1 · loading

Spinner'ı açar

2 · loaded

Veriyi state'e koyar

3 · failed

Hata mesajını gösterir

Thunk'ı bileşende kullanmak

```
function Users() {
  const { users, loading, error } = useSelector((s) => s.users);
  const dispatch = useDispatch();
  useEffect(() => { dispatch(fetchUsers()); }, [dispatch]);
  if (loading) return <p>Yükleniyor...</p>;
  if (error) return <p>Hata: {error}</p>;
  return <ul>{users.map((u) => <li key={u.id}>{u.name}</li>)}</ul>;
}
```

Bileşen sadece fetchUsers() dispatch eder. Yükleniyor, hata ve veri durumlarını state'ten okur.

Middleware ve akış

```
import { thunk } from "redux-thunk";
const store = createStore(usersReducer, applyMiddleware(thunk));
```



ALIŞTIRMA 3 · REDUX THUNK

Kullanıcı listesi çekelim

1. redux-thunk'ı kurun ve store'a bağlayın.
2. loading, loaded ve failed action'larıyla fetchUsers yazın.
3. Bileşende useEffect ile dispatch edin ve üç durumu ekrana yansıtın.

Süre: 30 dakika

Bonus: Yeniden dene butonu

ÖZET

Üç aracın karşılaştırması

Özellik	useReducer	Redux	Redux Thunk
Kapsam	Tek component	Tüm uygulama	Redux'a eklenen asenkron katman
State nerede?	Component içinde	Store'da	Store'da
Asenkron iş	Kendi başına yapmaz	Kendi başına yapmaz	Action creator içinde yapılır
Kurulum	React ile gelir	redux, react-redux	redux-thunk middleware
Çözdüğü sorun	Karmaşık local state	Prop drilling, paylaşılan state	Veri çekme ve yükleme durumları

HIZLI TEKRAR

Dört soru

1

Bir bileşen Redux state'ini nasıl değiştirir?

2

useSelector ve useDispatch ne işe yarar?

3

API çağrısını reducer içinde yapsak ne olur?

4

Thunk, normal bir action'dan nasıl ayrılır?

SONRAKI DERS

Redux Toolkit: aynı fikir, daha az kod

Bu derste elle yazdığımız switch'ler, createStore kurulumu ve thunk bağlantısı; Redux Toolkit ile configureStore, createSlice ve createAsyncThunk olarak çok daha kısa yazılacak.